

Using the Selection Monad

Gordon Plotkin

Types and Topology

A workshop in honour of Martín Escardó's 60th birthday

Smart binary search

```
let BinIndexSearch( $x$  : Int,  $a$  : Array[Int],  $l$  : Int,  $r$  : Int) =  
  let  $m$  : Int = choose-an-integer in [ $l$ ,  $r$ ]  
  if  $a[m] = x$  then  $m$   
  else cost(1); //pay-to-recurse  
    if  $a[m] < x$  then BinIndexSearch( $x$ ,  $a$ ,  $m + 1$ ,  $r$ )  
    else BinIndexSearch( $x$ ,  $a$ ,  $l$ ,  $m - 1$ )
```

A language

Types

$$\sigma ::= \text{Bool} \mid \text{Rew} \mid \dots \text{other basic types } \dots \mid \\ \text{Unit} \mid \sigma \times \sigma \mid \sigma \rightarrow \sigma$$

Terms

$$M_1 ::= x \mid \text{tt} \mid \text{ff} \mid 0 \mid \dots \text{other basic type constants } \dots \mid \\ M_1 = M_2 \mid M_1 \leq M_2 \mid M_1 + M_2 \mid \text{con}_p(M_1, M_2) \mid \dots \mid \\ \text{if } M \text{ then } M_1 \text{ else } M_2 \mid \\ M_1 \text{ or } M_2 \mid M_1 \cdot M_2 \mid M_1 +_p M_2 \mid \\ * \mid \langle M_1, M_2 \rangle \mid \text{fst}(M) \mid \text{snd}(M) \mid (\lambda x : \sigma. M) \mid M_1 M_2$$

Rewards

$$0 : \text{Rew} \qquad \frac{M_1 : \text{Rew} \quad M_2 : \text{Rew}}{M_1 \leq M_2 : \text{Bool}}$$

$$\frac{M_1 : \text{Rew} \quad M_2 : \text{Rew}}{M_1 + M_2 : \text{Rew}} \qquad \frac{M_1 : \text{Rew} \quad M_2 : \text{Rew}}{\text{con}_p(M_1, M_2) : \text{Rew}}$$

Effects: decision, reward, probability

$$\frac{M_1 : \sigma \quad M_2 : \sigma}{M_1 \text{ or } M_2 : \sigma} \qquad \frac{M_1 : \text{Rew} \quad M_2 : \sigma}{M_1 \cdot M_2 : \sigma} \qquad \frac{M_1 : \sigma \quad M_2 : \sigma}{M_1 +_p M_2 : \sigma}$$

The selection monad

The monad (due to Martín Escardó and Paulo Oliva)

$$S(X) =_{\text{def}} (X \rightarrow \mathbb{R}) \rightarrow X = X^{(\mathbb{R}^X)}$$

The elements of $S(X)$ are selection functions $F : (X \rightarrow \mathbb{R}) \rightarrow X$.

Given a reward function $\gamma : X \rightarrow \mathbb{R}$, F selects $F\gamma \in X$

This selection may be optimal:

$$F\gamma = \operatorname{argmax}_{x \in X} \gamma(x)$$

Unit Effect-free computations:

$$\eta_T(x) = \lambda \gamma. x$$

The reward associated to the selection $F\gamma \in X$ is:

$$R(F|\gamma) =_{\text{def}} \gamma(F\gamma)$$

Kleisli extension

$$\frac{X \xrightarrow{f} S(Y)}{S(X) \xrightarrow{f_S^\dagger} S(Y)}$$

Equivalently:

GIVEN $f : X \rightarrow S(Y)$ $F \in S(X)$ $\gamma : Y \rightarrow \mathbb{R}$ **FIND** $y \in Y$

Step 1 Find (optimal) reward for a given choice of $x \in X$:

$$\mathbf{R}(fx|\gamma)$$

Step 2 Find (best) choice of X for the resulting (optimal) reward continuation $x \mapsto \mathbf{R}(fx|\gamma)$:

$$x_{\text{best}} = F(x \mapsto \mathbf{R}(fx|\gamma)) : X$$

Step 3 Using γ , proceed with f with that choice of $x_{\text{best}} \in X$:

$$fx_{\text{best}}\gamma : Y$$

Which is to say:

$$f^{\dagger_S}(F)(\gamma) = fx_{\text{best}}\gamma = f(F(x \mapsto \mathbf{R}(fx|\gamma)))\gamma$$

Making binary choices

We can define a binary algebraic operation **or** for (binary) decision-making by making the choice giving the greatest reward. For $F, G \in S(X) = (\mathbb{R}^X \rightarrow X)$ set

$$(F \text{ or}_X G)\gamma = \begin{cases} F\gamma & (\text{if } \mathbf{R}(F|\gamma) \geq \mathbf{R}(G|\gamma)) \\ G\gamma & (\text{otherwise}) \end{cases}$$

This operation is associative and **left-biased**, meaning that each component or_X is, ie for all X :

$$(F \text{ or}_X G) \text{ or}_X H = F \text{ or}_X (G \text{ or}_X H)$$

$$F \text{ or}_X (G \text{ or}_X F) = F \text{ or}_X G$$

It is not commutative.

The selection monad transformer: adding other effects

Taking an **auxiliary** monad T to model other effects, one can combine it with the selection monad to obtain a monad

$$S(X) = (X \rightarrow \mathbb{R}) \rightarrow T(X)$$

One needs an algebra $\alpha : T(\mathbb{R}) \rightarrow \mathbb{R}$ to do this.

As auxiliary monad we take

$$DW(X) = \mathcal{D}_f(\mathbb{R} \times X)$$

Denotational semantics

- We combine DW with the selection monad to obtain:

$$S(X) = (X \rightarrow \mathbb{R}) \rightarrow \text{DW}(X)$$

- We then use Moggi's interpretation of the computational λ -calculus, to obtain a denotational semantics of our language:

$$\frac{\Gamma \vdash M : \sigma}{\mathcal{S}[\![\Gamma]\!] \xrightarrow{\mathcal{S}[\![M]\!]} S(\mathcal{S}[\![\sigma]\!])}$$

- To interpret binary decisions $M \text{ or } N$ we use the binary algebraic operator or_S on S , as (almost) defined above.
- To interpret rewards $r \cdot M$ and probabilistic choice $M +_p N$ we extend the corresponding DW-algebraic operations $(r \cdot -)_{\text{DW}}$ and $(+_p)_{\text{DW}}$ pointwise to S .

- As before

$$(F \text{ or}_X G)\gamma = \begin{cases} F\gamma & (\text{if } \mathbf{R}(F|\gamma) \geq \mathbf{R}(G|\gamma)) \\ G\gamma & (\text{otherwise}) \end{cases}$$

- but now

$$\frac{F\gamma = \sum_i p_i(r_i, \mathbf{x}_i)}{\mathbf{R}(F|\gamma) = \sum_i (r_i + \gamma \mathbf{x}_i)}$$

- Reward at DW level

$$r.\Sigma_i p_i(r_i, x_i) = \Sigma_i p_i(r + r_i, x_i)$$

- Reward at selection function level

$$(r.F)\gamma = r.(F\gamma)$$

- Probabilistic choice at selection function level

$$(F +_p G)\gamma = F\gamma +_p G\gamma$$

Ordinary operational semantics

Values

$$V ::= \text{tt} \mid \text{ff} \mid 0 \mid \dots \mid * \mid \langle V_1, V_2 \rangle \mid \lambda x : \sigma. M$$

Some redex transitions

$$r \leq r' \rightarrow \text{tt} \quad (\text{if } \llbracket r \rrbracket \leq \llbracket r' \rrbracket) \quad (\lambda x : \sigma. M[x]) V \rightarrow M[V]$$

$$M_1 \text{ or } M_2 \xrightarrow{\text{or}_i} M_i, \text{ for } i = 1, 2$$

$$r \cdot M_1 \xrightarrow[\cdot_1]{r} M_1, \text{ for } i = 1$$

$$M_1 +_p M_2 \xrightarrow[(+_p)_i]{} M_i, \text{ for } i = 1, 2$$

Program evaluation to effect terms

Effect values (= effect trees = interaction trees)

$$E ::= V \mid E_1 \text{ or } E_2 \mid r \cdot E \mid E_1 +_p E_2$$

Evaluation of programs to effect values

$$\text{Op}(M) = \begin{cases} V & (\text{if } M = V) \\ \text{Op}(M') & (\text{if } M \rightarrow M') \\ \text{Op}(M_1) \text{ or } \text{Op}(M_2) & (\text{if } M \xrightarrow{\text{or}_i} M_i, \text{ for } i = 1, 2) \\ r \cdot \text{Op}(M_1) & (\text{if } M \xrightarrow[\cdot 1]{r} M_1) \\ \text{Op}(M_1) +_p \text{Op}(M_2) & (\text{if } M \xrightarrow{(+_p)_i} M_i, \text{ for } i = 1, 2) \end{cases}$$

Effect terms E as games against nature

The positions of a game E are its subterms P :

- if P is a value V , then P is a final position, returning V ;
- if $P = \text{or}(P_1, P_2)$, **Player** can choose whether to move to position P_1 or P_2 ;
- if $P = r \cdot P_1 : \sigma$, **Player** moves to P_1 , and r is added to the accumulated reward (initially 0);
- if $P = P_1 +_p P_2$, **Nature** picks P_1 with probability p , and P_2 with probability $1 - p$.

Strategies, their outcomes, and their expected rewards

Game strategies

$$\begin{array}{c} * : V \\ \frac{s : E_1}{1s : \text{or}(E_1, E_2)} \quad \frac{s : E_2}{2s : \text{or}(E_1, E_2)} \\ \frac{s : E}{s : r \cdot E} \quad \frac{s_1 : E_1 \quad s_2 : E_2}{\langle s_1, s_2 \rangle : E_1 +_p E_2} \end{array}$$

Outcome distributions $\text{Out}(s, E) \in \mathcal{D}_f(\mathbb{R} \times \text{Val}_\sigma)$ for $E : \sigma$:

$$\begin{aligned} \text{Out}(*, V) &= \delta_{\langle 0, V \rangle} \\ \text{Out}(0s, E_1 \text{ or } E_2) &= \text{Out}(s, E_1) \\ \text{Out}(1s, E_1 \text{ or } E_2) &= \text{Out}(s, E_2) \\ \text{Out}(s, r \cdot E) &= r \cdot \text{Out}(s, E) \\ \text{Out}(\langle s_1, s_2 \rangle, E_1 +_p E_2) &= \text{Out}(s_1, E_1) +_p \text{Out}(s_2, E_2) \end{aligned}$$

Expected rewards

$$\mathbf{E} \left(\sum p_i \langle r_i, V_i \rangle \right) = \sum p_i r_i$$

The optimising operational semantics

Optimal strategy for a program $M : \sigma$

$$s_{\text{opt}} = \operatorname{argmax}(\lambda s : \operatorname{Op}(M). \mathbf{E}(\operatorname{Out}(s, \operatorname{Op}(M))))$$

Optimizing operational semantics $\operatorname{Op}_{\text{opt}}(M) \in \operatorname{DW}(\operatorname{Val}_{\sigma})$, for $M : \sigma$:

$$\operatorname{Op}_{\text{opt}}(M) = \operatorname{Out}(s_{\text{opt}}, \operatorname{Op}(M))$$

Two theorems

Adequacy

Theorem

For any program $M : b$ of basic type:

$$\mathcal{S}[[M]](0) = \text{Op}_{\text{opt}}(M)$$

Full Abstraction For $M, N : \sigma$ define observational equivalence by

$$M \approx_{\sigma} N \iff \forall C[] : \sigma \rightarrow \text{Bool}. \text{Op}_{\text{opt}}(M) = \text{Op}_{\text{opt}}(N)$$

Theorem

For any programs $M, N : b$ of basic type:

$$M \approx_b N \iff \mathcal{S}[[M]] = \mathcal{S}[[N]]$$

The problem

Smart Choices example: searching an ordered array a for a value x

```
BinIndexSearch( $x$  : Int,  $a$  : Array[Int],  $l$  : Int,  $r$  : Int) =  
  let  $m$  : Int a-choice-from [ $l$ ,  $r$ ] using [ $x$ ,  $a[l]$ ,  $a[r]$ ]  
  if  $a[m] = x$  then  $m$   
  else cost(1); //pay-to-recurse  
    if  $a[m] < x$  then BinIndexSearch( $x$ ,  $a$ ,  $m + 1$ ,  $r$ )  
    else BinIndexSearch( $x$ ,  $a$ ,  $l$ ,  $m - 1$ )
```

Combine two ideas

Handlers

+

Loss Continuations
(aka the selection monad)

=

Smart Handlers

Denotational Semantics - monads and types

The semantics is given assuming that the effects are well-founded.
This breaks the cycle in the following definitions:

Our selection monads

$$S_{\varepsilon}(X) = (X \rightarrow T_{\varepsilon}(\mathbb{R})) \rightarrow T_{\varepsilon}(\mathbb{R} \times X)$$

Our interaction trees: used to model unhandled operations

$$T_{\varepsilon}(X) = \left(\sum_{\substack{\ell \in \varepsilon \\ \text{op} : \mathbf{out} \xrightarrow{\ell} \mathbf{in} \\ 0 < i \leq \varepsilon(\ell)}} \mathcal{S}[\![\mathbf{out}]\!] \times T_{\varepsilon}(X)^{\mathcal{S}[\![\mathbf{in}]\!]}} \right) + X$$

Semantics of types

$$\mathcal{S}[\![A \rightarrow B! \varepsilon]\!] = \mathcal{S}[\![A]\!] \rightarrow S_{\varepsilon}(\mathcal{S}[\![B]\!])$$

Bibliography

Escardô and Oliva

Selection functions, bar recursion and backward induction

Mathematical structures in computer science 20.2 (2010): 127-168.

Escardô and Oliva

The Herbrand functional interpretation of the double negation shift

The Journal of Symbolic Logic 82.2 (2017): 590-607.

Abadi and Plotkin

Smart choices and the selection monad

Logical Methods in Computer Science 19 (2023).

Plotkin and Xie

Handling the Selection Monad (Full Version)

arXiv preprint arXiv:2504.03890 (2025).